Two Python Programming Deliverables: A Tested Hangman Game and a Regex-Based Email Extraction Automation Script

CodeAlpha Python Programming Internship — Tasks 1 & 3 (2 of 2 tasks completed)

SUMMARY

• Task 1 (Hangman Game): 5-word list, 6-guess limit, verified with 6 automated tests driving the real game loop.
• Task 3 (Email Extractor): regex-based extraction with case-insensitive dedup, tested against a real messy sample document.
• Both browser demos' JavaScript was cross-checked against the actual Python output and matched exactly.
• Hangman correctly rejects duplicate and invalid guesses without wasting a wrong-guess attempt.
• The email extractor correctly ignored 4 deliberately tricky non-email distractors in the real test document.
• Both projects include full source code, automated test suites, and standalone HTML demos.

Author: Mayank Swaraj (Mayur Krishna) | Mayank Creations

Program: CodeAlpha Virtual Internship — Python Programming Domain
Repository naming: CodeAlpha_HangmanGame, CodeAlpha_EmailExtractor
Date: July 2026 | Patna, Bihar, India

Part 1: Hangman Game (Task 1)

The brief specifies a simplified scope: 5 predefined words, no file or API dependency, 6 incorrect guesses allowed, and basic console I/O. This implementation satisfies exactly that scope, structured so the core game loop (play_game()) accepts injectable input/output/randomness functions, which is what makes automated testing of real gameplay possible rather than only testing isolated helper functions.

Design: Testable Game Loop

A common weakness in simple console games is that the game loop is tightly coupled to input()/print(), making it untestable without a real terminal. Here, play_game() takes input_func, print_func, and rng_choice as parameters (defaulting to the real input/print/random functions), so tests can drive the exact same logic used in real play with scripted, deterministic input.

Verification: 6 Automated Tests

Test	What it verifies
test_win_scenario	Guessing every correct letter wins with 0 wrong guesses
test_loss_scenario	Guessing 6 wrong letters loses with exactly 6 wrong guesses
test_duplicate_guess_does_not_count_twice	Re-guessing the same wrong letter only counts once
test_invalid_input_does_not_count_as_wrong	Multi-char/non-alphabetic input is rejected without penalty
test_display_progress	The underscore/letter display formats correctly at each guess state
test_word_list_composition	Exactly 5 words, all alphabetic, matching the brief's scope

All 6 tests pass. The duplicate-guess and invalid-input tests specifically guard against a common bug class in hangman implementations: silently double-penalizing a player for re-entering the same wrong letter, or for a typo.

Part 2: Email Extractor (Task 3)

Of the three automation options in the brief (moving .jpg files, extracting emails, scraping a webpage title), email extraction was chosen since it is fully self-contained and testable against a real, hand-crafted messy document without any filesystem-scanning or network dependency, while still automating a genuinely repetitive real task.

Extraction Pattern and Deduplication

```
EMAIL_PATTERN = r'[a-zA-Z0-9][a-zA-Z0-9._%+-]*@[a-zA-Z0-9][a-zA-Z0-9.-]*\.[a-zA-Z]{2,}
'
```

This is a practical (not RFC-5322-complete) pattern covering the vast majority of real-world addresses. Matches are deduplicated case-insensitively (domains and, in practice, most mail providers' local parts are treated as case-insensitive), keeping the first-seen casing for display.

Verification Against a Real, Deliberately Messy Document

Rather than testing only clean, obviously-valid email strings, a realistic sample document was constructed containing 7 genuine emails, one deliberate case-variant duplicate (Sarah.Jones@Company.com vs. sarah.jones@company.com), and 4 non-email distractors designed to trip up a naive regex: a version number (v2.3@build), a social-media-style @mention, text missing the @ symbol entirely, and a string

missing the required dot in the domain (almost@valid).

Test	Result
Real sample document (7 real emails, case-variant dup, 4 distractors)	All 7 correctly extracted and deduplicated; all 4 distractors correc
Case-insensitive deduplication (isolated)	Confirmed: 2 case-variant inputs -> 1 output
Rejects non-emails (isolated)	Confirmed: 0 false positives on 5 non-email strings
Handles +/- characters in local part/domain	Confirmed: urgent+oncall@my-company.io extracted correctly
Multiple emails on one line	Confirmed: all 3 extracted in order
Empty/no-email input	Confirmed: returns an empty list, no crash

Browser Demo Cross-Verification

Both HTML demos reimplement their Python counterpart's exact logic in JavaScript. For the email extractor, the actual deployed JavaScript function (extracted from the HTML file and run directly in Node.js, not a hand-copied approximation) was run against the same real sample document used to test the Python script, and produced an identical list of 7 emails in the identical sorted order — confirming the browser demo is a faithful mirror, not just a similar-looking reimplementation.

Conclusion

Both deliverables satisfy their respective briefs' simplified scopes while going beyond a minimal implementation: the Hangman game's testable architecture enables real automated verification of actual gameplay sequences, and the email extractor was verified against a realistic, deliberately adversarial test document rather than only clean inputs. Both browser demos were confirmed to produce output identical to their Python source.

References

[1] Python Software Foundation. re — Regular expression operations. docs.python.org.

[2] CodeAlpha. (2026). Python Programming Tasks & Instructions. www.codealpha.tech.

“A script that only works on clean input hasn't been tested — it's been demonstrated.”

— Mayank Swaraj